

BURSA TEKNİK ÜNİVERSİTESİ

BLM230 Bilgisayar Mimarisi Dönem Projesi

Hazırlayan: Arif Kemal Şeremet

Öğrenci No: 24360859049

Demo Video Linki: <https://youtu.be/Q-SghrYZq4s>

1. Projenin Amacı ve Kapsamı

Bu projenin temel amacı, veri iletimi ve depolanması sırasında oluşabilecek tek bitlik hataların tespiti ve düzeltilmesi işlemlerini görsel ve etkileşimli bir şekilde simüle etmektir. Geliştirdiğimiz Hamming Hata Düzeltme Kodu Simülatörü, 8, 16 ve 32 bitlik veri boyutları için kodlama ve hata düzeltme süreçlerini adım adım göstermektedir. Takım çalışmasına dayalı kolektif bir çabanın ürünü olan bu projede, bit-flip (bit tersleme) senaryoları ile yapay hatalar oluşturarak, sendrom hesaplama algoritmalarının nasıl çalıştığını detaylı bir arayüzle görselleştirmeyi hedefledik.

2. Simülatörde Kullandığımız Teknolojiler

Projemizde ana programlama dili olarak Python'ı, kullanıcı arayüzü (UI) tasarımı için ise Tkinter kütüphanesini tercih ettik. Python'ın esnek yapısı karmaşık matematiksel işlemleri sadeleştirirken, Tkinter sayesinde kullanıcıların bitler üzerinde doğrudan etkileşime (bit-flip) girebileceği dinamik bir sistem geliştirdik. Özellikle 32-bit veri gösteriminde karşılaştığımız arayüz taşma (overflow) sorununu, Tkinter'in Canvas bileşenini kaydırılabilir (scrollable) bir yapıda entegre ederek takımca çözüme kavuşturduk. Ayrıca, kodlamanın anlaşılabilir olması adına parite hesaplamalarını ve XOR işlem zincirlerini ekranda görselleştirdik.

3. Hamming Kodu Matematiksel Altyapısı

Hamming kodlarında orijinal veriye eklenecek olan kontrol (parity) bitlerinin sayısını belirlemek için $2^r \geq m + r + 1$ eşitsizliğini temel aldık. Bu eşitsizlikte m iletilen mesaj (veri) bitlerinin sayısını, r ise eklenecek parite bitlerinin sayısını temsil etmektedir. Geliştirdiğimiz sistem, girilen veri uzunluğuna göre bu formülü arka planda işleterek gereken r değerini otomatik olarak bulmaktadır. Örneğin 4 bitlik bir veri ($m = 4$) için formül $2^3 \geq 4 + 3 + 1$ denklemini sağladığından, gereken parite biti sayısı $r = 3$ olarak tayin edilir.

4. Parity Pozisyonları ve Matematiksel Seçim

Hesaplanan parite bitleri, oluşan yeni veri kümesinde her zaman 2'nin kuvvetleri olan pozisyonlara (1, 2, 4, 8, 16...) yerleştirilir. Veri bitleri ise boş kalan diğer alanlara sırasıyla dizilir. Her parite biti, ikilik (binary) tabandaki indeksinde kendi pozisyon değerine denk gelen bitleri (XOR ile) kontrol eder:

- **P1 (1. bit):** İndeksinin en sağındaki biti 1 olan pozisyonları kontrol eder (1, 3, 5, 7, 9, 11...).
- **P2 (2. bit):** İndeksinin sağdan ikinci biti 1 olan pozisyonları kontrol eder (2, 3, 6, 7, 10, 11...).
- **P4 (4. bit):** İndeksinin sağdan üçüncü biti 1 olan pozisyonları kontrol eder (4, 5, 6, 7, 12, 13...).

5. Örnek Çözüm: "0111" Verisinin Kodlanması

Girdi verimiz olarak 0111 ($m=4$) ele alındığında, formüle göre $r=3$ parite biti gerekir. Toplam iletim uzunluğu 7 bit olur.

Pozisyon	1	2	3	4	5	6	7
Bit Türü	P1	P2	D1	P4	D2	D3	D4
Veri Yerleşimi	?	?	0	?	1	1	1

Arayüzümüzün de görselleştirdiği hesaplama adımları şöyledir:

- $P1 = D1 \oplus D2 \oplus D4 = 0 \oplus 1 \oplus 1 = 0$
- $P2 = D1 \oplus D3 \oplus D4 = 0 \oplus 1 \oplus 1 = 0$
- $P4 = D2 \oplus D3 \oplus D4 = 1 \oplus 1 \oplus 1 = 1$

Oluşan Hamming Kodu: 0 0 0 1 1 1 1

6. Sendrom Hesaplama ve Hata Tespiti (Compare Modülü)

Veri iletiminin ardından sistemimizde manuel olarak arayüz üzerinden bir bit tıklanarak (bit-flip) yapay bir hata oluşturulmaktadır. **Compare Modülümüz**, alıcı tarafında tüm parite değerlerini yeniden hesaplar ve gelen orijinal parite bitleri ile karşılaştırır. Bu karşılaştırma işleminden çıkan sonuç "Sendrom Kelimesi"ni (Syndrome Word) oluşturur. Sendrom değerinin onluk (decimal) karşılığı, doğrudan hatalı bitin pozisyonunu vermektedir. Bu XOR zincirini ve oluşan hata tespit kodlarını uygulamamızın arayüzünde şeffaf bir şekilde kullanıcılara sunduk.

7. Hatanın Düzeltilmesi (Corrector Modülü)

Sendrom değeri hesaplanıp hatalı bitin pozisyonu kesin olarak tespit edildikten sonra **Corrector Modülü** devreye girer. Bu modül, tespit edilen indeksteki bitin durumunu (1 ise 0'a, 0 ise 1'e) tersine çevirerek bozulan biti orijinal haline döndürür. Böylece veri bütünlüğü yeniden sağlanmış olur. Simülatörümüz bu adımı anlık bir animasyon veya renk vurgusu ile göstererek, hatanın tespit edilip anında düzeltildiği süreci dinamik bir şekilde ekrana yansıtır.